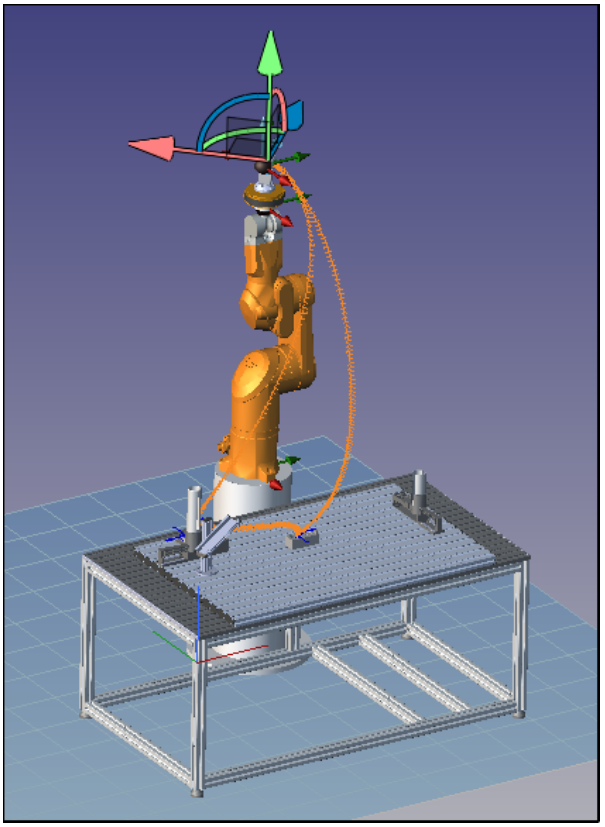
Robot Manipulation

Tutorial about

STÄUBLI ROBOTICS SUITE 2019

and VAL3



STÄUBLI®
ROBOTICS

ATTENTION: The robot is a fast moving machine. These movements can be dangerous. Before moving the robot, always check that the robot's environment is in the expected state and make sure that no one is inside the robot's working space.

1. Introduction

This paper aims to explain the basic operation of the Stäubli Robotics Suite software (2019). This is a 3D programming and simulation environment for Stäubli robots. In addition, it has the ability to emulate the corresponding control cabinets. This simulation is performed through the CS8 controller which is integrated into the Stäubli Robotics Suite 2019 software.

2. Start SRS 2019

In order to start the Stäubli Robotics Suite 2019 (SRS 2019) software, you must double-click on the desktop shortcut.



Figure 1: Icon to start the Stäubli Robotics Studio 2019 software.

Once the software has been initialized, the following window will appear (see Figure 2), in which we will have to choose whether to create a new work cell or whether to open an existing one. Work cells simulate an environment where there are 1 or more robots, and all the relevant objects in that environment to be able to do a proper simulation.

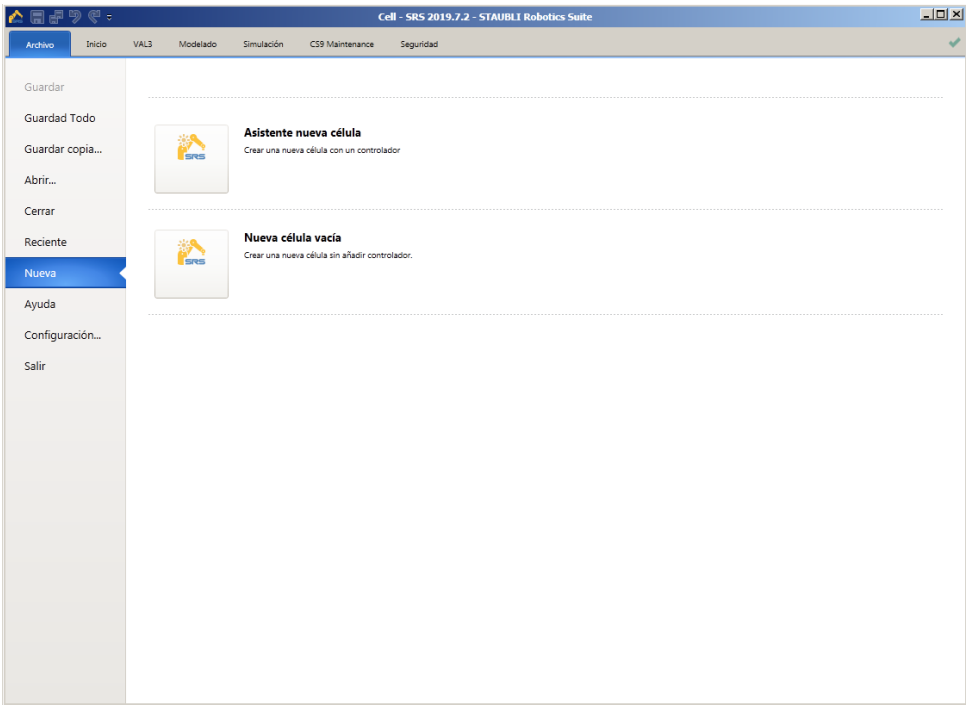


Figure 2: Initial window of SRS 2019 to choose the cell in which we want to work.

3. Options bar

Once we have a cell loaded, we can make different choices about it. The options bar allows us to access 5 main tabs: File, Start, VAL3, Modeling, and Simulation. (see Figure 3).



Figure 3: Tools on the "Start" tab.

3.1 Home tab

This tab (see Figure 3) contains the main tools that allow you to perform the necessary actions to add new robots to the cell and view it in 3D. Also to manage the controller and the emulator as

well as all their respective connections. In addition, it allows you to open the license manager. In the event that we have accidentally closed any of the work windows, it also allows us to deploy the desired windows again. Finally, through the “Window” menu, “Window Distributions”, it is possible to manage the distribution of windows and save specific configurations (see Figure 4).

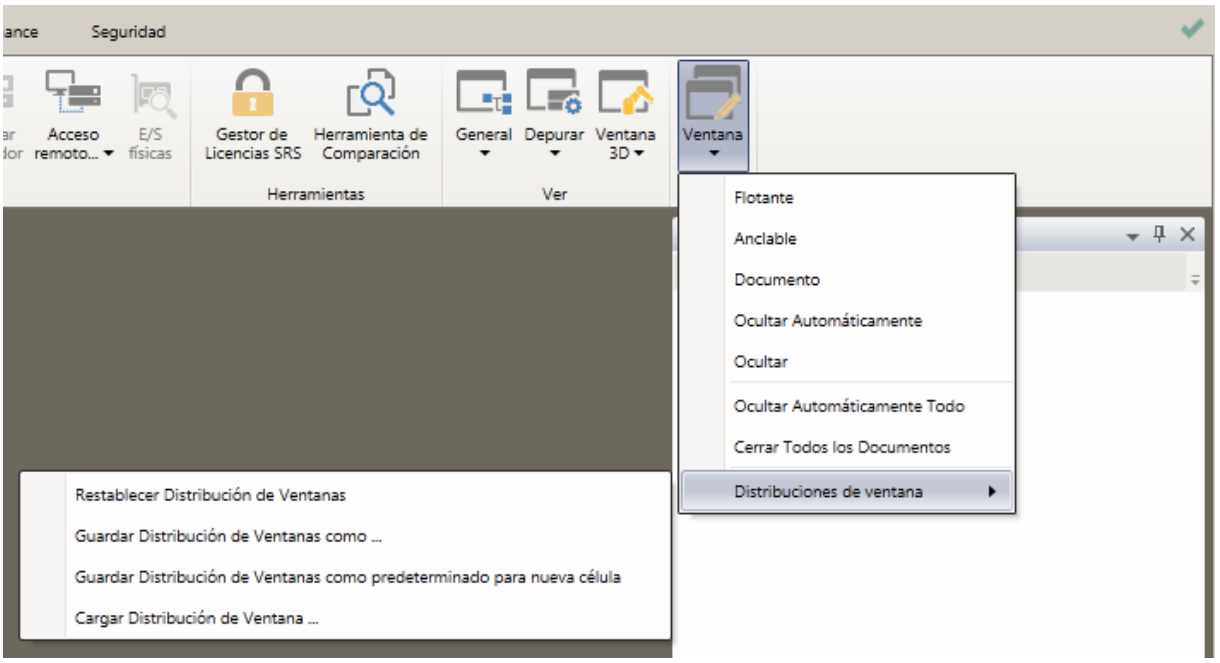


Figure 4: Path to save or open a specific window distribution.

3.2 VAL3 tab

This tab (Figure 5) allows you to create and manage VAL3 applications. Contains options for syntax checking, debugging and running applications, ... From this tab you can also add libraries.

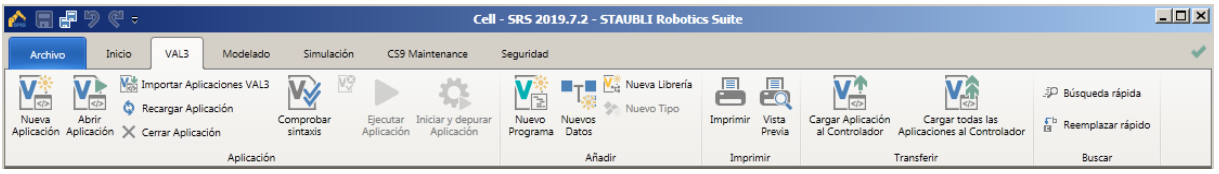


Figure 5: Tools on the VAL3 tab.

3.3 Modeling tab

This tab (Figure 6) allows you to manage 3D geometries. Geometries created through other CAD software can be added or basic geometries can be generated. It also allows you to define the positions of these geometries. If desired, the "Tools" submenu allows you to add a geometry as a tool, editing its characteristics.



Figure 6: Modeling tab tools

3.4 Simulation tab

This tab (Figure 7) contains all the tools related to 3D simulation. From here you can select the type of movement to which you want to subject the robot and edit points through the position of the robot. It also allows you to generate a set of points to show the trajectory made by the robot and make collision detection. Finally, it contains the sync menu that allows you to sync multiple controllers with the same pulse train.



Figure 7: Tools on the Simulation tab

Locate each of the tabs listed and look at the tools each has.

4. Cell editing environment

Apart from the options allowed by the SRS discussed in the previous section, the different elements of the cell are visualized and edited using the environment shown in Figure 8.

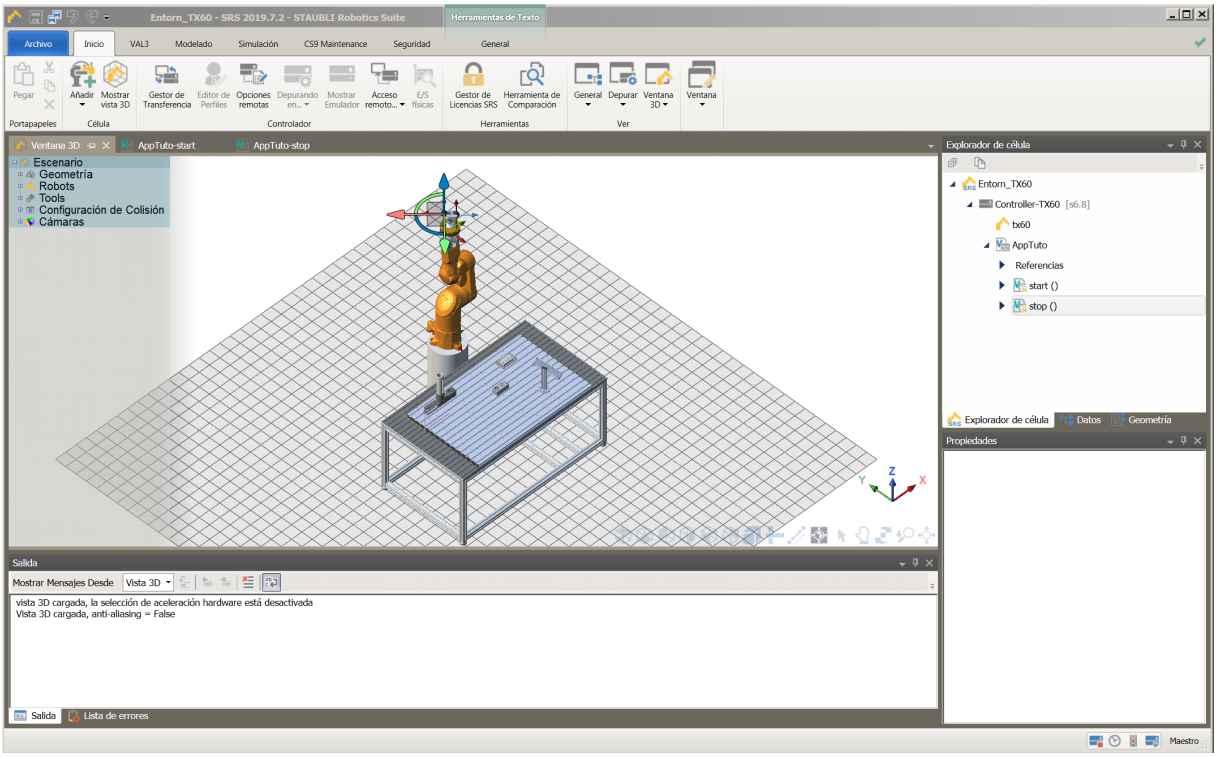


Figure 8: Cell editing environment.

There is a set of windows that show the following information:

- *3D Window: Displays a view of the 3D environment of the cell. Contains tools for moving the viewpoint, zooming in or out, and selecting objects. It also contains a menu that includes all the objects present in the environment and their typology.*
- *Cell Explorer: Window showing the cell controllers (or robots) and open applications with their program contents and libraries.*

- Program editing window: When we select (double click) a program (for example the "start" program) within an application in the cell browser, it can be viewed and edited in the same area where the 3D window, in another tab.
- *Data: When we go to the Data tab we see all the application variables organized by type. From this tab you can modify its values by double-clicking on one of them. New data can also be created there.*
- *Geometry: When we go to the Geometry tab we see all the points, tools and coordinate systems of the application. We can view its value or modify it.*
- *Properties: This window shows the properties of the object that we have selected in the previous menus. It also allows you to view and modify values.*
- *Output: This window displays the messages that the SRS emits. These are organized into three categories: general, 3D view, and emulator.*
- *Error List: This tab displays a list of messages, errors, and warnings that are generated during syntax checking, debugging, and running an application.*

Locate each of the listed windows and tabs.

5. Cell load

Once the software has started, unzip the “Environment_TX60.zip” file and open the “Environment_TX60.cell” cell. You will see the environment in Figure 8, in which the practice environment of the TX60 manipulator in the laboratory is represented.

6. Creating VAL3 applications

6.1 Introduction

To create a VAL3 application you need a work cell and a controller with an associated robot. As you can see in Figure 8, we already have the cell created and the controller. Now, therefore, we can create a new VAL3 application.

6.2 Assigning the new application to a driver

Next we will create an application associated with the TX60 robot which is associated with the controller called "Controller-TX60". To create the application, right-click on the driver name and, from the drop-down menu that opens, click on "New application".

After clicking "New application" a pop-up window appears (Figure 9) which allows you to edit the name of the application, its location and finally allows you to start with a specific template (we will not use it). In our case we will leave the location and the template that already comes out by default and call it AppTuto.

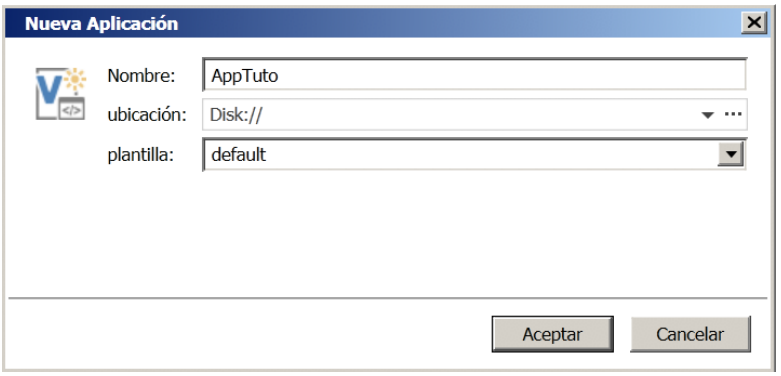


Figure 9: New application configuration pop-up window

As can be seen in Figure 10, the application created is associated with the controller corresponding to the TX60 robot.

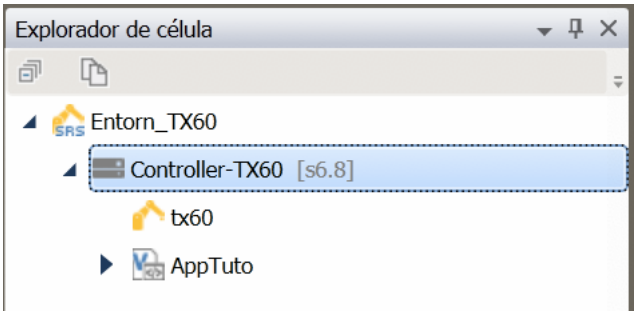


Figure 10: Cell browser with the new application associated with the TX60 robot controller

6.3 Content of the new application in the cell explorer

The new application contains two routines called "start" and "stop". The "start" routine is the one that runs automatically when you start the application. The "stop" routine is the one that is executed at the end.

On the other hand, as can be seen in Figure 11, the created application also contains a drop-down list of all inputs and outputs (io) that can be used in the application.

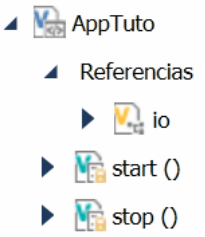


Figure 11: Visualization of the content of the generated application

6.4 Content of the new application in Datos

As shown in Figure 12, the created application has also appeared in the "Data" tab. This contains the data corresponding to this application. In this case, we can see how there is the variable called mNomSpeed, which contains the default value of the arm speed.

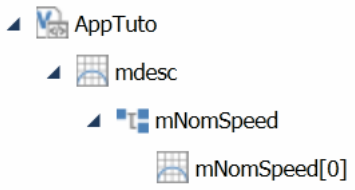


Figure 12: Structure of the generated application data

To add new data, right-click on the name of the application, select "add", Nuevos datos (see Figure 13).

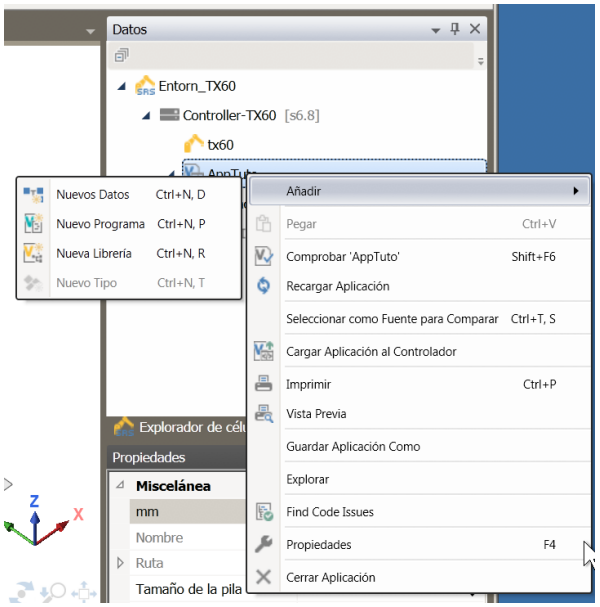


Figure 13: Insertion of new data from the generated application

Then a pop-up window will appear (Figure 14) where you can select the type of data to be added. Depending on whether it is geometric data or not, it will be added to "Data" or "Geometry" (often both). So let's add a new non-geometric data, for example a variable (a number).

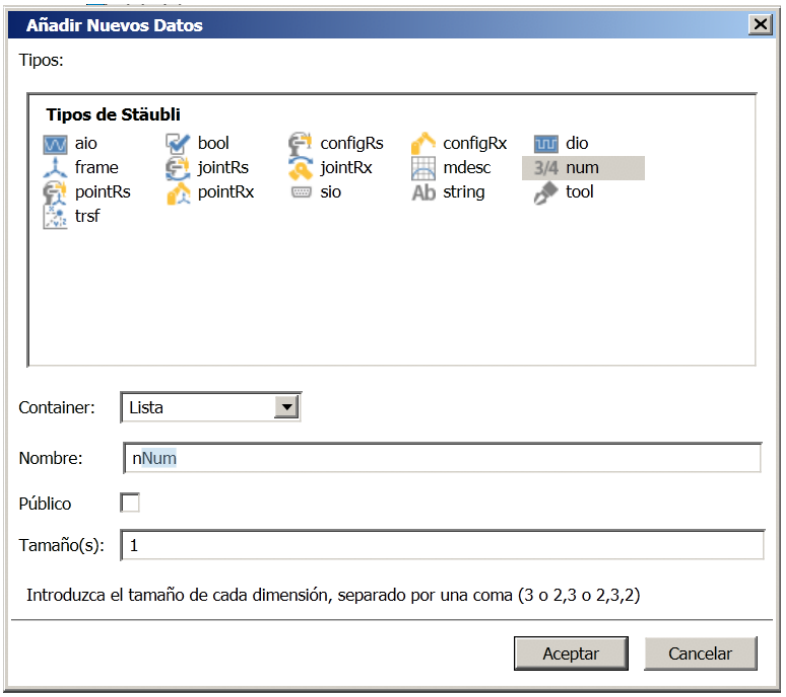


Figure 14: Window that allows you to select the type of data to be added to an application

If we configure the new data as shown in Figure 14 and click accept, we see how this new data, which is of type num, appears in the "Data" tab.

As can be seen in Figure 15, the non-geometric data are grouped according to the data type. So for now we have two groups: mdesc, which contains the data that holds the values of the arm speeds; and num, which contains data that stores numeric values.

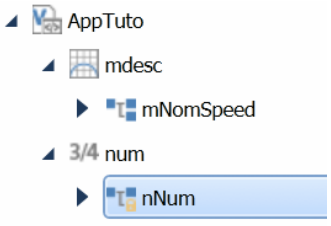


Figure 15: Organization of the data in the "Data" tab by data type

6.5 Content of the new application in Geometry

As you can see in Figure 16, the created application has also appeared in the Geometry tab. As already mentioned, inside this tab is all the geometric data of the application.

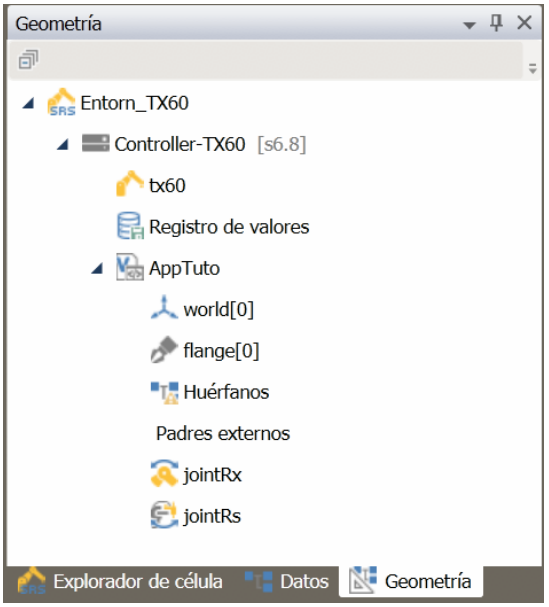


Figure 16: Organization of the data in the "Geometry" tab by type of geometric data

By repeating a process similar to the one explained above, new geometric data can be added both from the application and from a specific data type. In both cases you have to right-click and select "New Data". Then a window opens where you can select the type of data, edit its name, size, etc.

6.6 Editing the generated application (Start and Stop)

In order to edit the AppTuto app, we will edit the Start and Stop routines. As mentioned above, the Start routine is the one that is executed at the start of the application and the Stop routine is executed at the end of it.

If we go to the cell browser and double-click on the Start () or Stop () routines (Figure 11), we see that a new tab opens in which we can directly edit each of these routines (Figure 17).

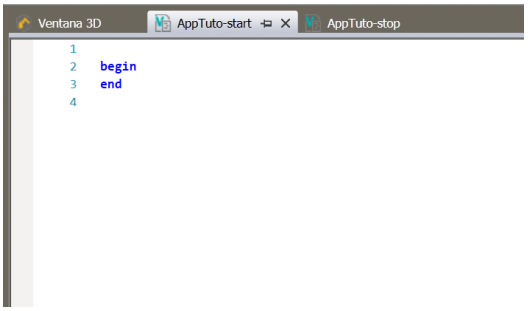


Figure 17: Tabs for editing Start () routines

We also notice that a new tab called “Text Tools” appears in the options bar (see Figure 18).

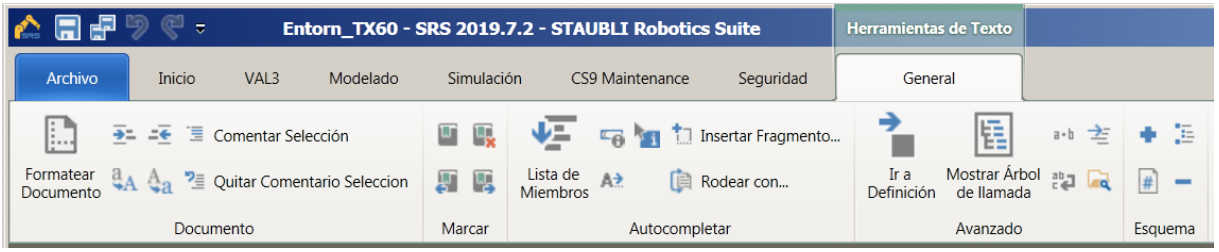


Figure 18: Option bar with the new “Text Tools” menu selected

We will use the following code which must be copied into the Start () and Stop () routines:

Start ():

```
begin
userPage()
cls()
putln("Start the program")
enablePower()
mNomSpeed.vel=30
movej(position2[0], eina0[0], mNomSpeed)
close(eina0[0])
putln("Position 2a reached. Terminal element closed.")
open(eina0)
movej(position2[1], eina0[1], mNomSpeed)
close(eina0[1])
```

```
putln("Position 2b reached. Terminal element closed.")
end
```

Stop ():

```
begin
movej(position0, eina0, mNomSpeed)
open(eina0)
putln("Position 0 reached. Terminal element open. Program completed")
popUpMsg("Pending movement commands have been canceled")
resetMotion()
end
```

These commands correspond to a very simple code which is used to show how to use the software. This tutorial is not dedicated to explaining the code.

The elements that will have to be defined so that the application can be compiled are: position0, position2, tool0, mNameSpeed. We will edit them one by one below:

1. position0: go to data, right-click, “add”, “New data” (Figure 29). We then set up the new point as shown in Figure 19.

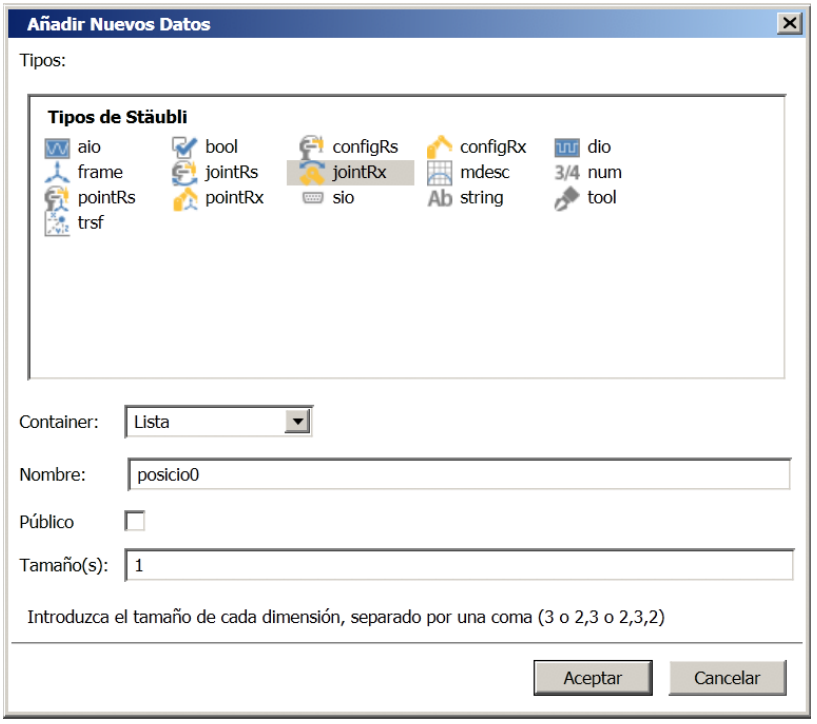


Figure 19: editing the "position0" point, of the jointRx type (joint positions)

2. position2: we repeat the previous process and configure the new data (pointRx type point) as shown in Figure 20.

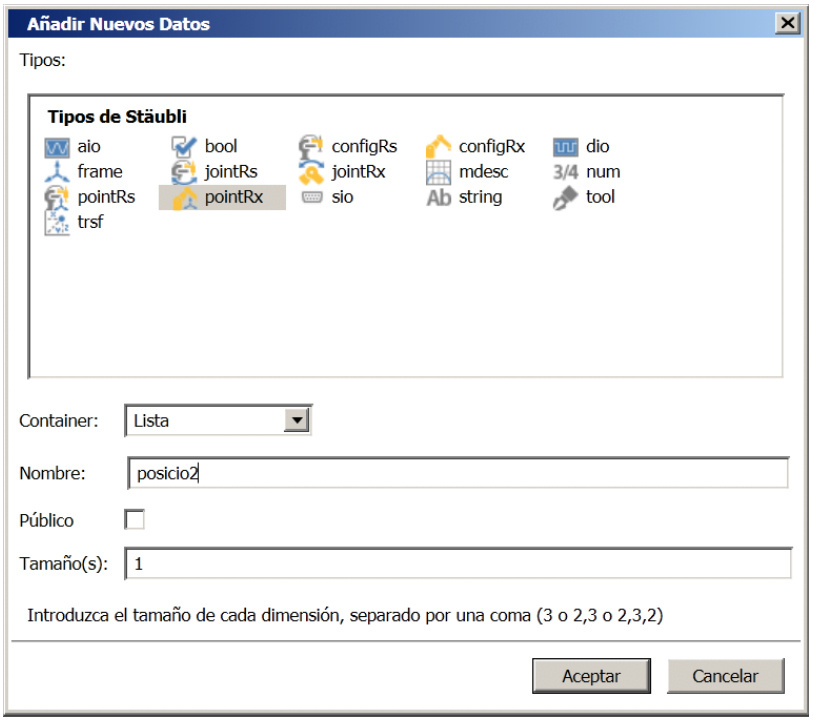


Figure 20: editing the "position2" point, of type pointRx (Cartesian position)

3. tool0: we repeat the previous process and configure the new data (tool type tool) as shown in Figure 21.

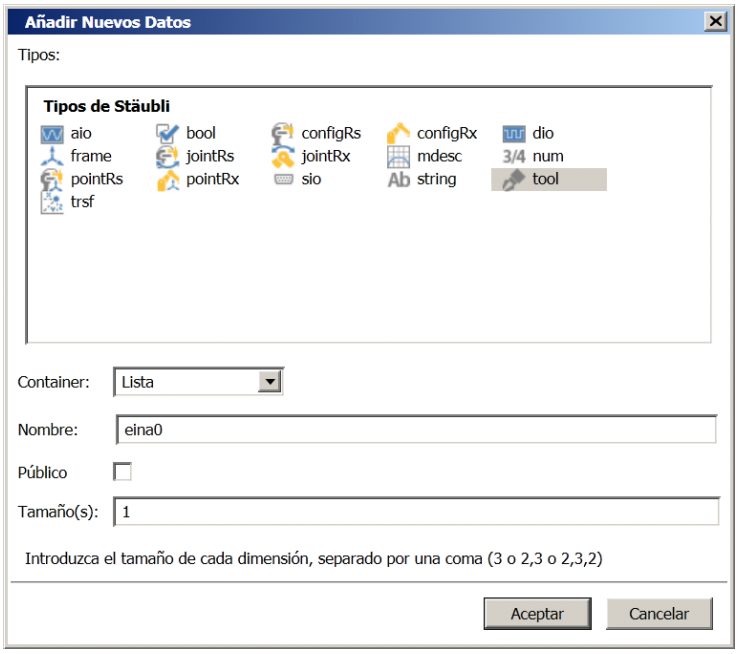


Figure 21: Editing the "tool0" tool, of the tool type

4. mNameSpeed: Default is already set when creating the application.

In Figure 22 we see what the "Data" tab looks like after declaring all the data needed to run the edited "AppTuto" application.

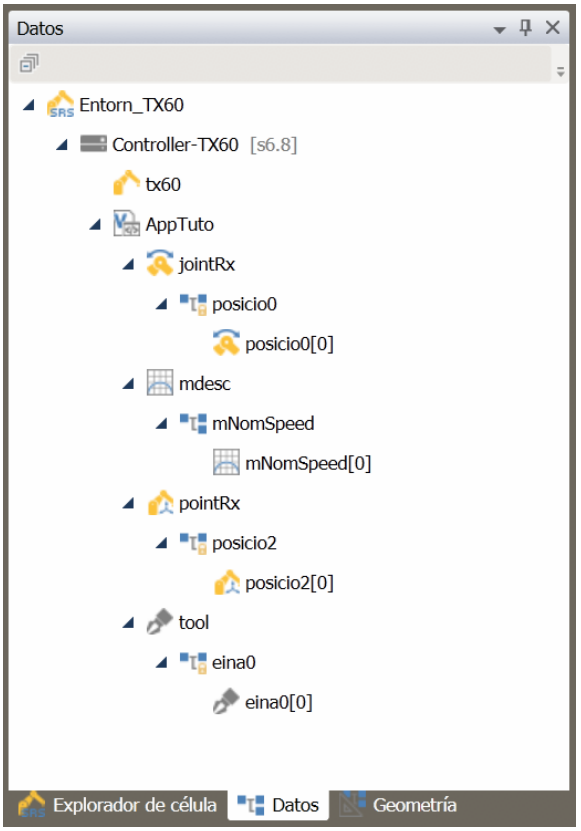


Figure 22: structure of the data declared in the "Data" tab

Next, let's check the code. To do this, we close the code editor tabs. When asked if we want to save it and say yes (we could have previously saved it with the floppy disk icon or by pressing Ctrl + S). Then we go to the cell browser, click on the AppTuto app and, in the options bar inside the VAL3 tab, click check syntax (see Figure 23).



Figure 23: Syntax checking

We can observe (Figure 24) that there is no error or warning that alerts us to any error in editing the code.

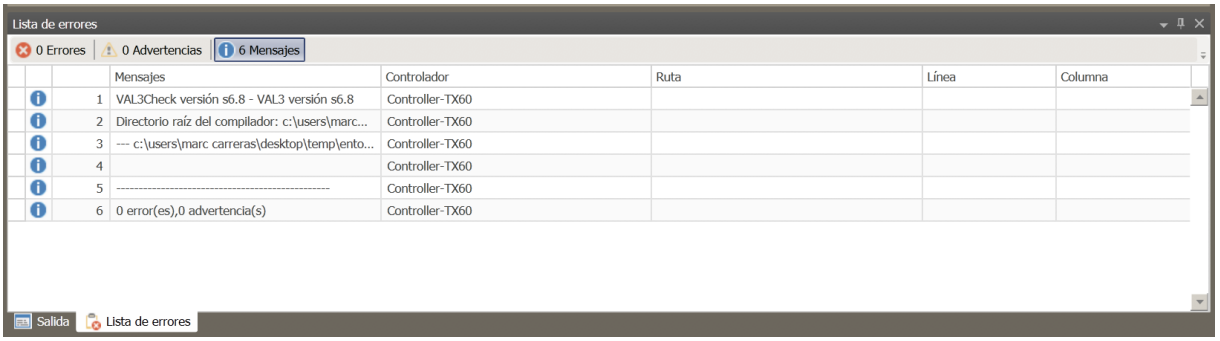


Figure 24: List of errors shown when checking the syntax of a code

Finally, with the necessary data created and the code written, we already have the application generated. All that remains is to simulate it through the emulator and link it to the 3D work environment.

Before that, however, since it will be necessary to explain the emulator, it is briefly discussed how the profile editor works.

6.7 Editing points and the tool

Previously, within the data tab, two points and a tool have been declared. Next we will edit your positions and features. We will start with the point called “position0” of the jointRx type, we will

continue with the point called “position2” of the pointRx type and, finally, we will end with the tool called “tool0” which is of the tool type.

Each of these elements should be defined specifically as each of them has different characteristics. For example, joint-type points are defined through the angular positions of each of the robot’s degrees of freedom. On the other hand, point-type points are defined through the Cartesian positions and the angular position of the coordinate system corresponding to the point. Finally, the tool should also be defined similarly to point-type elements. The software, however, allows you to geometrically edit each of these points and then assign the geometry to the elements already defined, whatever their type.

In the options bar, in the submenu “Construction plans” (see Figure 25), we can find the tabs needed to be able to edit points within the 3D scenario. These can be assigned to the data already defined previously in the application.

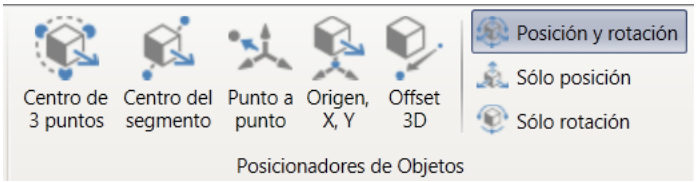


Figure 25: Tabs for editing points within 3D geometry

This process will then be performed for each of the three elements required to run the sample program:

1. Point "position0": this point is the rest of the robot. It is of the joint type and corresponds to all the angular positions equal to zero. As can be seen in 3D geometry, the robot is already in this position. In addition, the default values of the point are already with all angles equal to zero. Therefore, it is recommended not to make any changes.
2. Point “position2”: this element contains two points inside it. When we defined it it was done in such a way that it was one-dimensional. As the sample code is written this

element must be two-dimensional. To increase its size we go to the "Data" tab and select it.

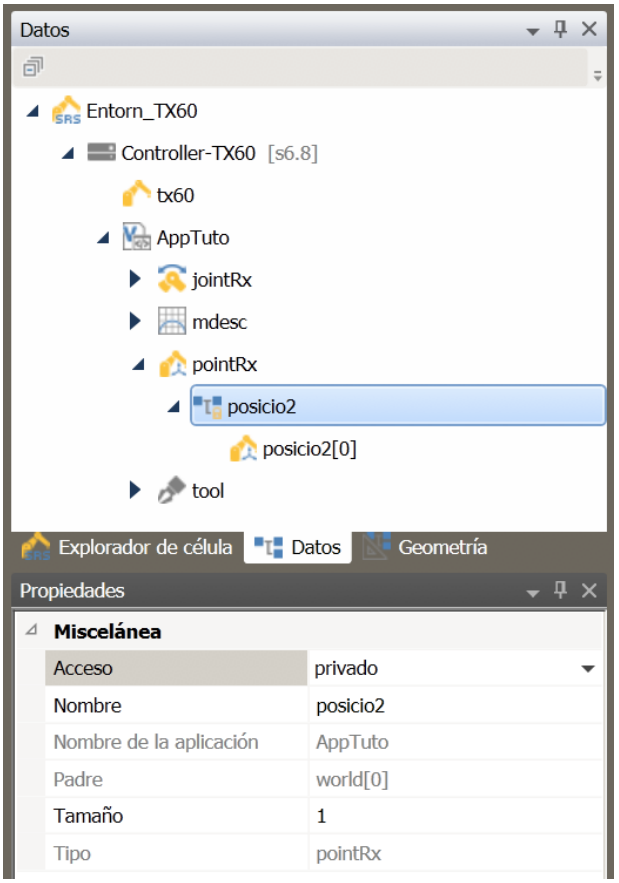


Figure 26: Editing the pointRx type dimension called position2

As you can see in Figure 26, inside the properties tab we see that the "Size" of the point is 1. We proceed to modify this value indicating that we want it to be two. In Figure 27 you can see how the point called “position2” is no longer one-dimensional and now contains two elements. These can be edited and their values changed. If we double-click on the point, a new tab will open (Figure 28) in which we can edit the values of each of the points.

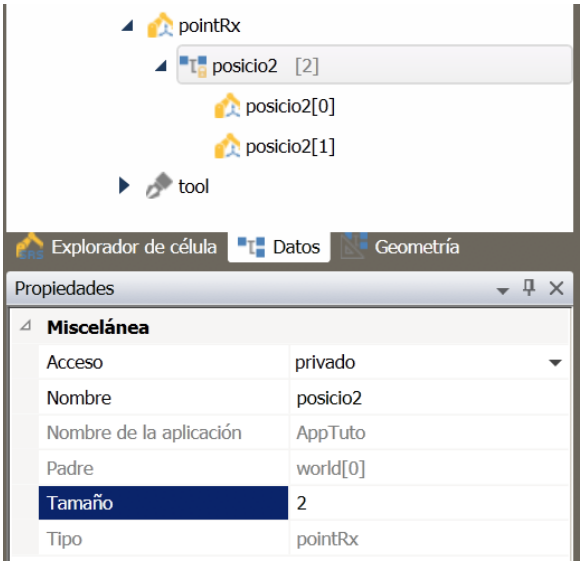


Figure 27: visualization of the two-dimensional frame which contains 2 elements type pointRx

Datos AppTuto-posicio2* | Ventana 3D* | AppTuto-start | AppTuto-stop

Índice	X	Y	Z	Rx	Ry	Rz	Shoulder	Elbow	Wrist
0	0	0	0	0	0	0	ssame	esame	wsame
1	0	0	0	0	0	0	ssame	esame	wsame

Figure 28: List of variables that make up a pointRx type point

Then we just define the position of the point by editing its absolute values. We introduce the values that can be seen in Figure 29.

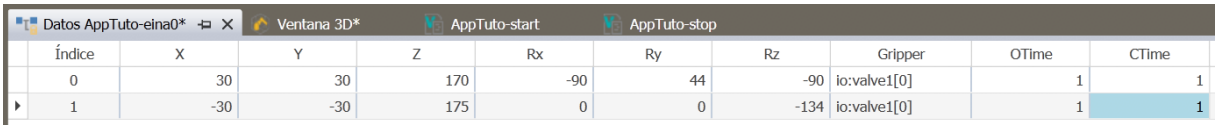
Datos AppTuto-posicio2 | Ventana 3D | AppTuto-start | AppTuto-stop

Índice	X	Y	Z	Rx	Ry	Rz	Shoulder	Elbow	Wrist
0	379,50001	-289,5	-285,18	-180	0	90	sfree	efree	wfree
1	546,99999	-61,00001	-265,18	180	0	-90	sfree	efree	wfree

Figure 29: Absolute position of the position position2.

Make sure that the configuration parameters: shoulder, elbow and wrist, have the free option, in order to allow the CS8 controller maximum freedom to reach the positions.

3. Tool “tool0”: in order to include the geometric characteristics of the tool in the “Data” tab, it is necessary to edit the geometric transformations that characterize the tool (see Figure 30). In this case we have two tools, as with the same mechanism you can take pieces of different dimensions depending on the side used.



Índice	X	Y	Z	Rx	Ry	Rz	Gripper	OTime	CTime
0	30	30	170	-90	44	-90	io:valve1[0]	1	1
1	-30	-30	175	0	0	-134	io:valve1[0]	1	1

Figure 30: Geometric characteristics of the tool0, together with the address of the associated digital output and time of opening and closing in seconds.

Edit these values so that both tools are set correctly.

No tools need to be selected to check jointRX positions. However, for pointRx positions, you must first select which tool you want. So select tool0 [0], expand the menu and click “Set As Current Tool”, then select position2 [0] and click “Move to”. To reach position2 [1] you need to do the same procedure but defining tool0 [1] as the current tool.

Finally, the fully defined elements are available. The generated code can now be executed without any alarm signal being generated.

7. Emulator

The emulator is the software responsible for simulating the behavior of the control cabinet of the robot, specifically its MCP[1]. To be able to run it, right-click on the controller and select “Show emulator” (see Figure 31).

[1] MCP: Manual Console Programming.

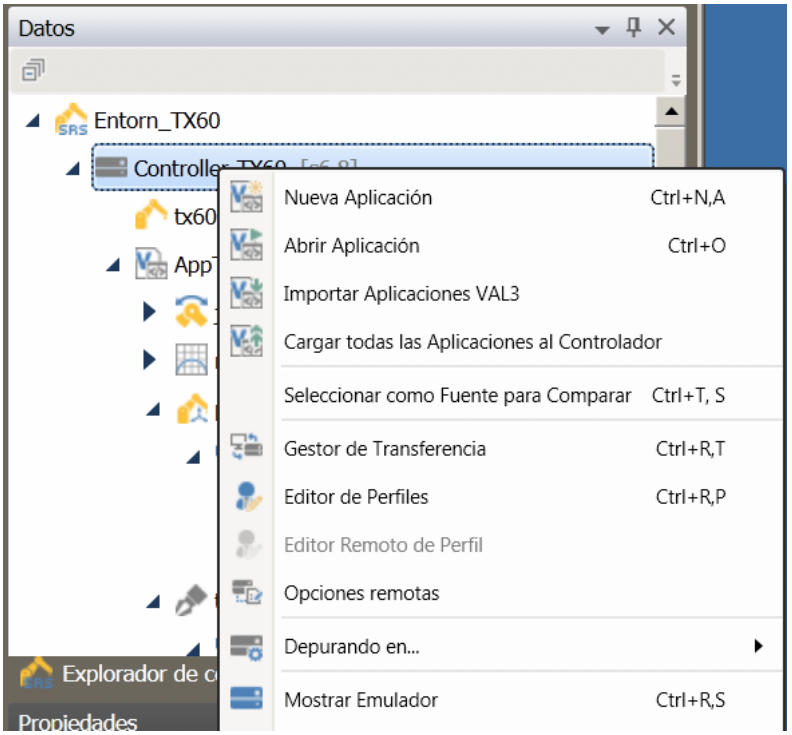


Figure 31: Path to activate the control cabinet MCP emulator

Once we have clicked “Show Emulator”, a pop-up window will appear (Figure 32). Which is responsible for simulating the behavior of the MCP of the control cabinet. The figure shows the Main Menu display.

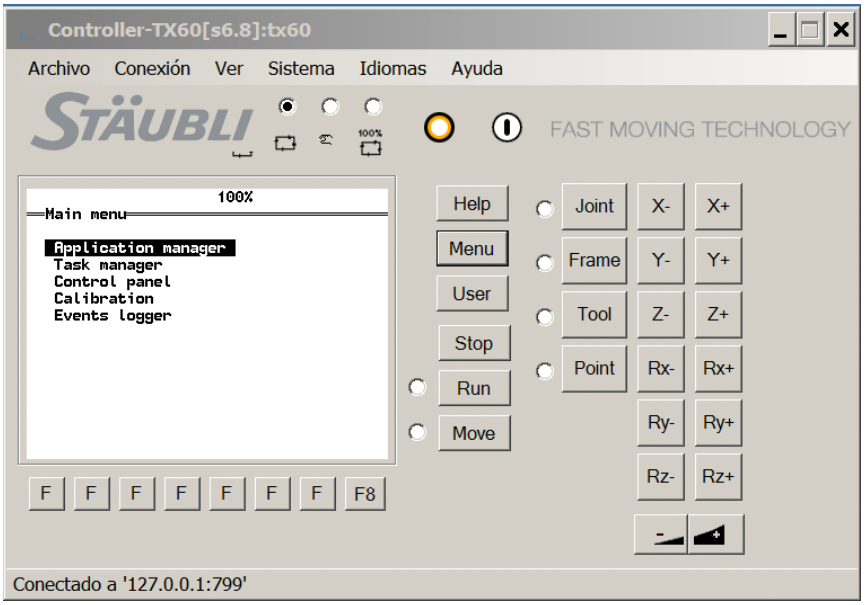


Figure 32: CSP emulator of the CS8 control cabinet

The arrows on the keyboard allow us to scroll through the menus of the emulated MCP. The arrow on the right allows us to select the menu in bold. To return to the main menu, use the cursor to click the menu key.

Then we will open the AppTuto application to simulate its execution. To do this, enter the menu "Application manager", "Val3 Applications", "Disk" (Figure 33).

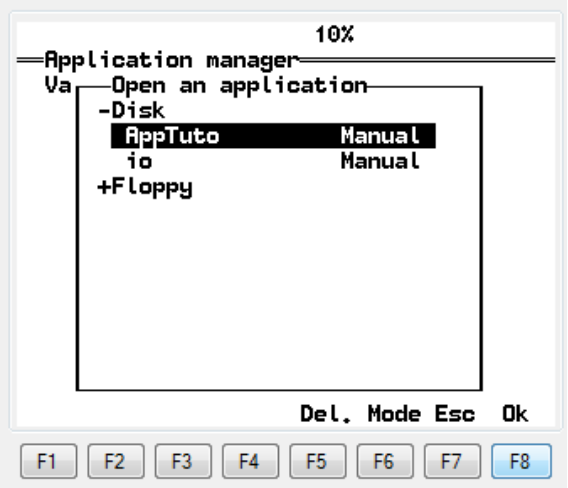


Figure 33: Opening the AppTuto application through the emulator

After clicking Ok, select "Menu". Finally, we are ready to start the simulation.

8. 3D simulation

To perform the simulation it is necessary that the emulator is activated and synchronized with the 3D environment. To this end we go to the options bar, simulation within the synchronization subgroup (Figure 34).

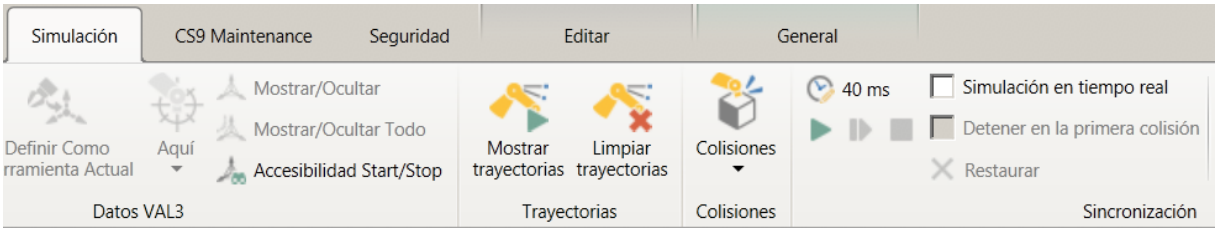


Figure 34: Option bar with the simulation tab selected

As can be seen in Figure 34, it is recommended to select a synchronization time of 40 ms. This is fast enough to visualize the simulation without crashing the computer. In order for the execution

of a program to be synchronized with the emulator and the 3D environment, the play symbol must be activated.

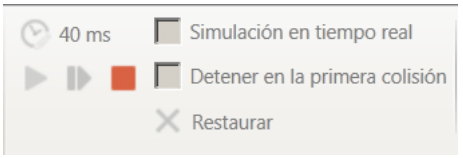


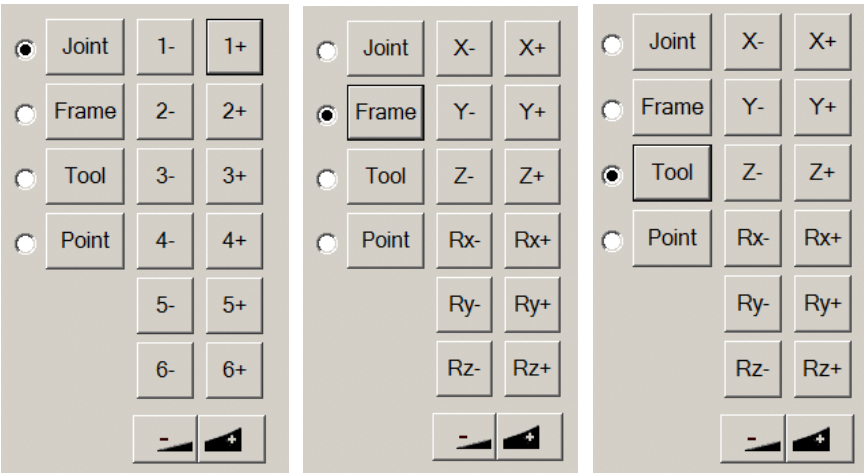
Figure 35: Simulation tab synchronization submenu

As can be seen in Figure 35, the play symbol is light gray when activated.

Return to the emulator and select Manual mode using the button  .

Once selected  , power the arm using the button  . You may get an error stating that power cannot be given because the MCP must be removed and replaced. This is a safety measure that forces us to show that the MCP is located on its support and that it is away from the work area of the manipulator. Go to "System" uncheck "Parked" and check it again. Give power again. Now you should not see the error and the power button should have changed color to green  .

Once you are in manual mode, and have power in your arm, you can move the robot with “Joint”, “Frame” and “Tool” modes. Select these modes and move the robot using the keys:



Check the movement of the robot in the 3D Window and change the viewpoint and the number of views. Draw the traces of the robot's movement on the screen.

Change the speed  of the movements in the emulator.

In Frame mode, find out the arrangement of the X, Y, and Z axes in the figure.

We are now in a position to simulate the behavior of the robot with the generated code. In order to be able to visualize the trajectory of the robot, we will select the "show traces" tab (Figure 34). Next we go to the CS8 emulator and click the Run key. A tab will then open inside the screen of the emulated MCP (Figure 36) which indicates the applications that are loaded in the emulator. Since we have loaded the AppTuto app, it appears in the list. Select it using the arrows on the keyboard and click Ok.

Once this process is done, we will give power to the arm so that the movement can begin. To do this we click the button until it turns green  . This tells us that the arm already has power.

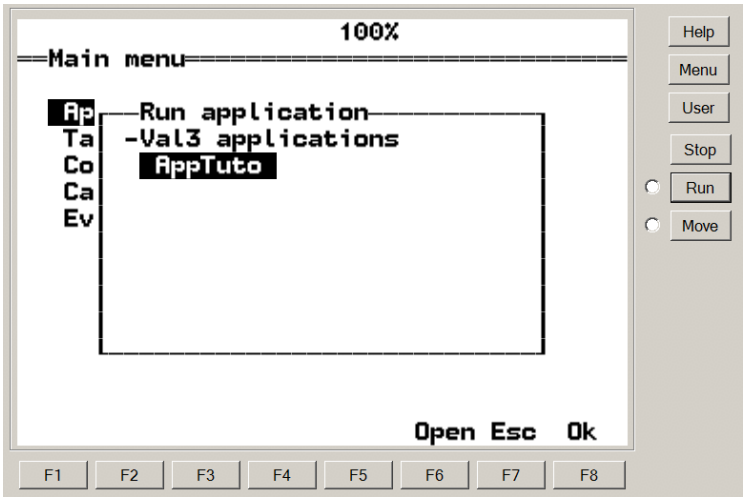


Figure 36: Selection of the application to be simulated via the Run key

Finally, we are ready to start the movement. To do this, click the "Move" key on the emulator. The result at the end of the simulation is as follows:

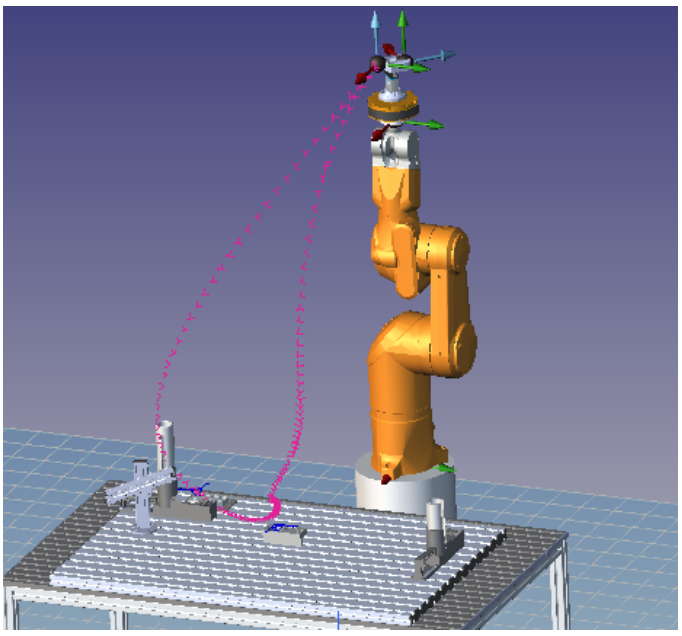


Figure 37: trajectory performed by the simulation.

9. Collision detection

During the movement of the robot, there may be collisions between the geometries. Within the simulation the rupture of any of the parts is not generated, but in reality, a collision can lead to the total rupture of the robot. For this reason, the software incorporates the possibility of detecting collisions during the simulation. To activate this option, click on the tab called “Collisions” (Figure 34).

If we perform the simulation again repeating the process followed above, during the simulation the areas where the collisions are located are visualized. Modify the coordinates of the position2 variable so that it collides with the environment.

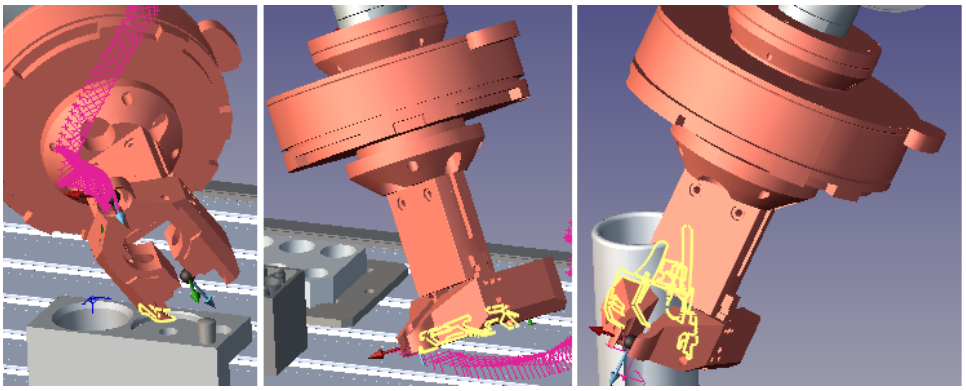


Figure 38: Visualization of collisions during simulation

10. Transfer manager

When the application is ready in the simulation field, it can be loaded to the robot. The Transfer Manager is a tool incorporated in the SRS software. It allow to transfer the applications generated in the simulation environment to the Controller (CS8C) which controls the real robot. You can find it in the options bar, in the principal grupo, inside the subgrup controller (see Figure 39).

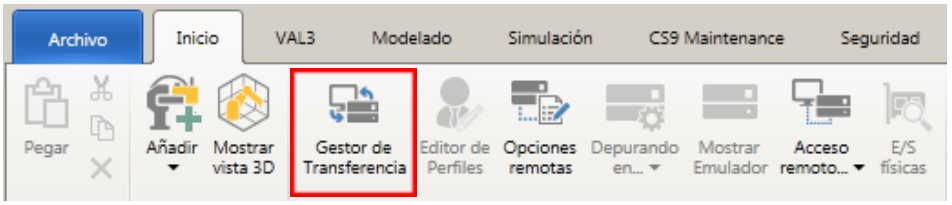


Figure 39: Transfer Manager button ubication.

This tool requires a network connection to which the actual driver is connected. In the case of the laboratory, a local network is already installed where all the computers are connected to the CS8 control cabinet of the robot.

To proceed with transferring an application to the actual control cabinet, select the driver for our robot (menu on the right) and click on the "Transfer Manager" tab.

A new popup window will then appear (see Figure 40). This allows you to edit the IP of the control cabinet, the port and the user profile. We will leave the default user "default" and port "5653" and use the following IP according to the manipulator used in the "Host" field:

Manipulator	IP
TX60 - 6 axis	84.88.129.201
TS60 - Scara	84.88.129.200

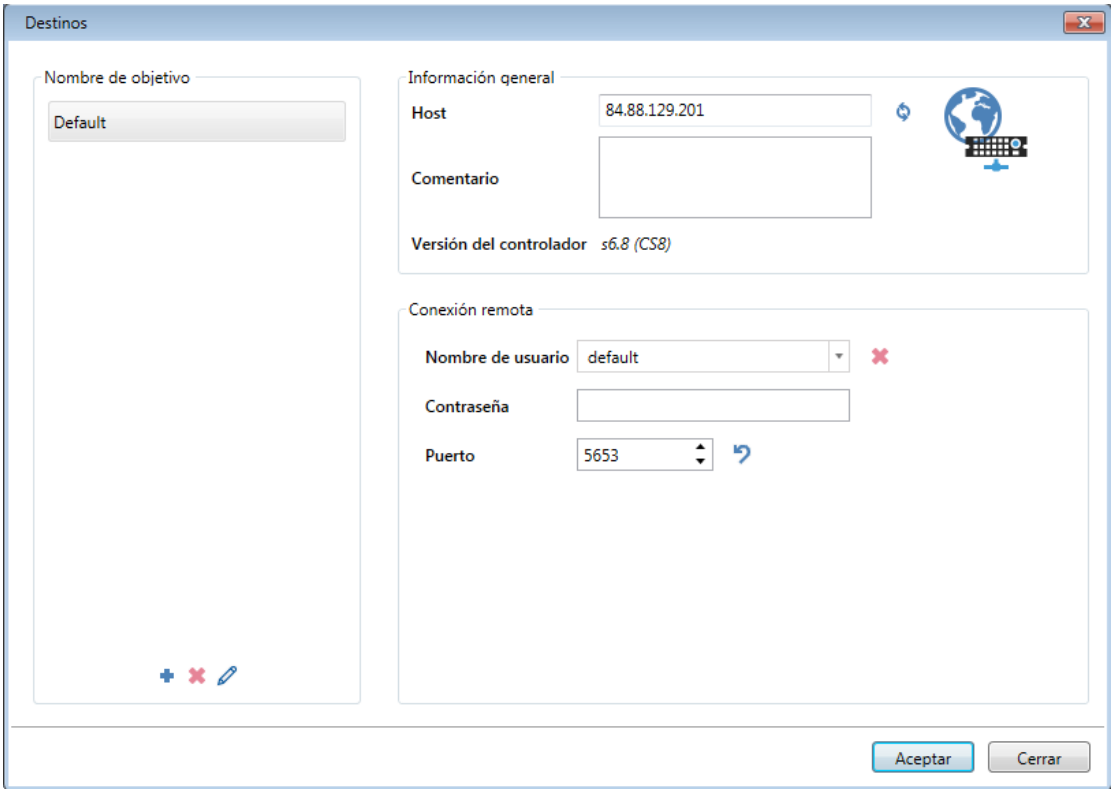


Figure 40: Window for edit the IP of the real controller.

Once the computer is connected to the control cabinet, we are ready to transfer our application to the control cabinet. To do this, select the entire application folder in the left window, and click the arrow to send it to the right, which displays the contents of the manipulator's CS8 controller (see Figure 41):

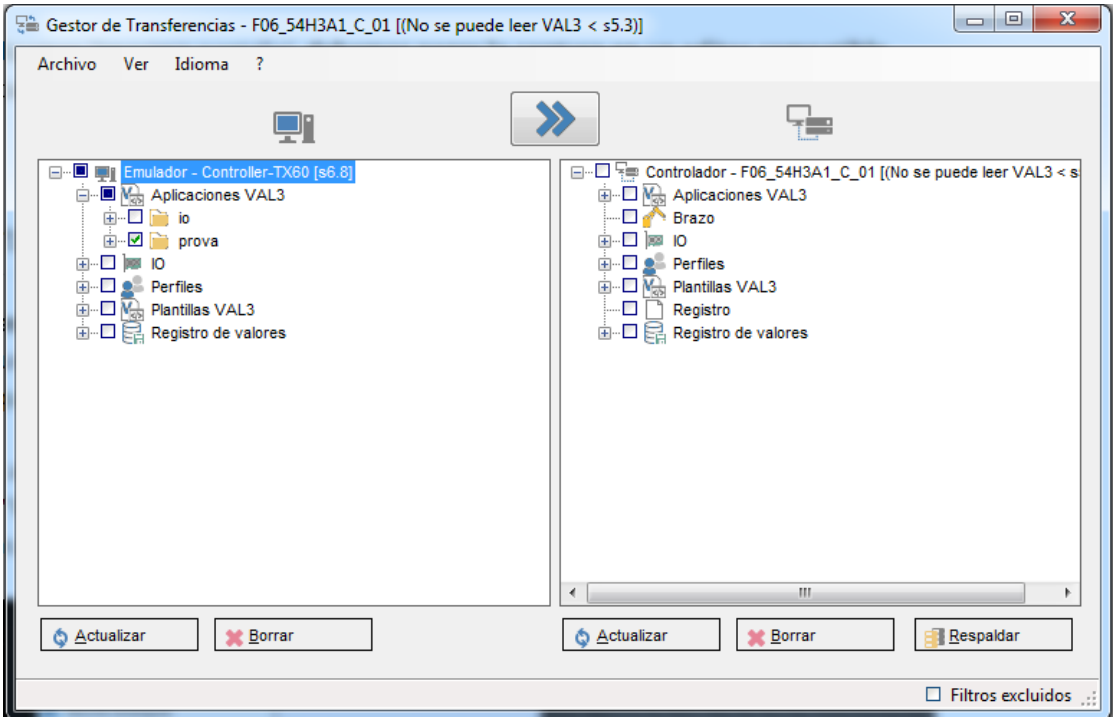


Figure 41: Selection and transfer of the VAL3 application.

Once the application has been transferred, we can check that it is already in the file structure of the CS8 driver, and we can now close the transfer manager. If we go to the MCP of the actual control cabinet, we will see that the new application appears on the screen when we enter the application manager.

IMPORTANT: Once you have made any changes from the MCP, such as saving new positions, you will have to resend the application with the transfer manager, making the transfer in the opposite direction (from the right menu to the left menu).